

DOCUMENTO TÉCNICO DE DISEÑO.

Proyecto integrador – Plataforma e-commerce Ecommify

Dataset: Brazilian E-commerce (Olist)

Enfoque arquitectónico: Arquitectura Políglota

EQUIPO DE TRABAJO

Camilo José Mora Rodríguez 0000394950

Carlos Alberto Zambrano Braydi 0000395349

Alejandro Miguel López Castañeda 0000385228

Carlos Arturo Salazar Castañeda 0000393238

Asignatura: EFMAS-DOBD20263

Mayo 2026

Abstract

El presente documento consolida el diseño técnico de la base de datos para Ecommify, plataforma de comercio electrónico multivendedor basada en el dataset real Brazilian E-commerce (Olist). El proyecto adopta una arquitectura políglota híbrida que distribuye la carga de datos entre dos motores especializados según la naturaleza de cada módulo.

PostgreSQL gestiona el núcleo transaccional del sistema: órdenes de compra, pagos, clientes y vendedores. Esta decisión responde a la necesidad de garantizar integridad referencial estricta y transacciones ACID multi-tabla, fundamentales en operaciones monetarias y de inventario. Se incorporan tipos de datos avanzados como JSONB para metadatos de pasarela de pago, arrays para almacenamiento de contactos múltiples y composite types para soporte multi-moneda. La extensión pg_trgm complementa el esquema con búsquedas tolerantes a errores tipográficos.

MongoDB maneja el módulo analítico y de catálogo: productos, reseñas, geolocalización y comportamiento de usuarios. Su esquema flexible permite representar atributos heterogéneos por categoría de producto y escalar horizontalmente las lecturas masivas sin afectar el núcleo transaccional. El modelo relacional fue normalizado hasta 3FN, garantizando eliminación de redundancias y dependencias transitivas. La separación de responsabilidades entre motores, sustentada en el Teorema CAP, permite que una eventual falla en el dominio analítico no comprometa las operaciones críticas del negocio.

1. Análisis de requisitos

1.1 Contexto de Ecommify y requisitos funcionales/no funcionales.

Ecommify es una plataforma de comercio electrónico multivendedor orientada a productos tecnológicos, opera bajo un modelo en el que múltiples vendedores independientes publican su catálogo y los clientes realizan compras que pueden combinar productos de distintos orígenes en una misma orden, el sistema debe gestionar de forma eficiente tanto el flujo transaccional (registro de compras, procesamiento de pagos, seguimiento de entregas) como el flujo analítico (catálogo de productos, reseñas de clientes, distribución geográfica).

Para el desarrollo del proyecto se utiliza el dataset Brazilian E-commerce (Olist), compuesto por nueve tablas en formato CSV que representan la operación real de una plataforma de e-commerce brasileña entre 2016 y 2018.

Requisitos funcionales

- Registrar y gestionar clientes con su información de contacto y ubicación.
- Administrar los vendedores vinculados a la plataforma y su zona de operación.
- Crear órdenes de compra y hacer seguimiento de su estado durante todo el proceso (creación, aprobación, envío y entrega).
- Registrar el detalle de cada orden: qué productos incluye, qué vendedor los despacha, precio y costo de flete.
- Procesar los pagos de cada orden permitiendo múltiples métodos y pago en cuotas.

- Registrar las reseñas que los clientes dejan después de recibir su pedido, incluyendo calificación y comentario.
- Consultar el catálogo de productos con sus características y categoría.
- Manejar la información geográfica de clientes y vendedores mediante código postal.

Requisitos no funcionales

- El sistema debe responder de forma rápida, apuntando a tiempos de respuesta por debajo de los 100 ms en lecturas.
- La arquitectura debe poder crecer sin que eso implique rediseñar todo desde cero.
- Una falla en el módulo analítico (MongoDB) no debe afectar las operaciones de compra y pago que corren sobre PostgreSQL.
- Los datos monetarios deben manejarse con precisión decimal, sin aproximaciones que afecten los valores reales de los pagos.
- El sistema debe soportar múltiples usuarios conectados al mismo tiempo sin degradar el rendimiento.

1.2 Identificación de entidades del dominio.

A partir del análisis del dataset Olist se identifican las siguientes entidades principales, clasificadas según el motor de base de datos que las gestionará.

Módulo transaccional — PostgreSQL

- Customers: Compradores registrados en la plataforma con su ubicación y datos de contacto.
- Sellers: Vendedores asociados responsables de preparar y despachar los pedidos.
- Orders: Entidad central que registra cada transacción de compra con su estado y fechas de ciclo de vida.
- Order_Items: Detalle de los productos incluidos en cada orden, con precio unitario y costo de flete.
- Order_Payments: Métodos y valores de pago utilizados para liquidar una orden.

Módulo analítico — MongoDB

- Products: Catálogo de artículos con dimensiones físicas y categoría. Sus atributos varían según el tipo de producto.
- Category_Translation: Tabla de referencia que traduce los nombres de categorías del portugués al inglés.
- Order_Reviews: Retroalimentación del cliente tras la entrega, con calificación numérica y comentario en texto libre.
- Geolocation: Catálogo de códigos postales con coordenadas geográficas para estandarizar ubicaciones.

1.3 Tabla de volúmenes de datos esperados.

Los volúmenes de datos se obtienen directamente del análisis exploratorio ejecutado sobre el dataset Olist en Google Colab, estos valores sirven como línea base para el dimensionamiento de la arquitectura.

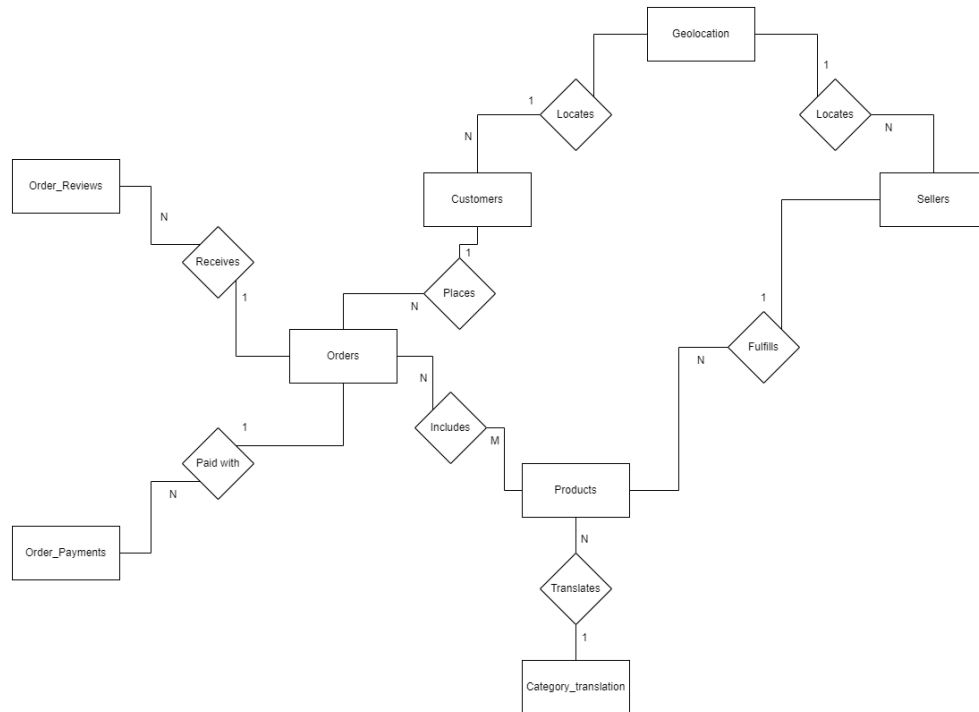
Para la tabla inserta 5 columnas en Word: **Tabla / Registros actuales / Duplicados / Motor asignado / Crecimiento estimado** y llena así:

- olist_customers_dataset → 99.441 → 0 → PostgreSQL → Alto
- olist_orders_dataset → 99.441 → 0 → PostgreSQL → Alto
- olist_order_items_dataset → 112.650 → 0 → PostgreSQL → Alto
- olist_order_payments_dataset → 103.886 → 0 → PostgreSQL → Alto
- olist_sellers_dataset → 3.095 → 0 → PostgreSQL → Medio
- olist_order_reviews_dataset → 99.224 → 0 → MongoDB → Alto
- olist_products_dataset → 32.951 → 0 → MongoDB → Medio
- olist_geolocation_dataset → 1.000.163 → 261.831 → MongoDB → Bajo
- product_category_name_translation → 71 → 0 → MongoDB → Muy bajo

La tabla geolocation es la de mayor volumen con más de un millón de registros y presenta 261.831 duplicados por código postal, lo que refuerza la decisión de moverla a MongoDB con índices geoespaciales y una estrategia de deduplicación en la carga inicial.

2. Diseño Conceptual

2.1 Diagrama ER completo de alta calidad.



2.2 Descripción detallada de cada entidad y relación.

2.2.1 Entidades del Módulo Transaccional.

- **Customers:** Registra a los compradores de la plataforma. guarda un identificador único y su zona de residencia.
- **Sellers:** Contiene los vendedores asociados que preparan y despachan los productos.
- **Orders:** Es la entidad central en la que se registra la compra, la cual tendrá un estatus e información de fechas para aprobación, envío y entrega.
- **Order_Payments:** Registra los métodos de pago, número de cuotas y valor utilizado para saldar una orden de compra.

2.2.2 Entidades del Módulo no relacional.

- **Products:** Catálogo de los artículos ofrecidos, incluyendo sus dimensiones físicas.

- **Category_Translation:** Entidad que guarda la información de las categorías traducidas del portugués al inglés.
- **Order_Reviews:** Entidad de retroalimentación que registra la satisfacción del cliente después de la entrega, almacenando su calificación y comentarios.
- **Geolocation:** Catálogo de códigos postales con sus coordenadas, ciudades y estados para estandarizar las ubicaciones de clientes y vendedores.

2.2.3 Entidades del Módulo no relacional.

- **Places [1:N]:** Un cliente puede realizar múltiples órdenes a lo largo del tiempo, pero una orden pertenece a un solo cliente.
- **Includes [N:M]:** Una orden puede incluir múltiples productos y un producto puede ser vendido en múltiples órdenes.
- **Fulfills [1:N]:** Un vendedor se encarga de despachar múltiples productos vendidos en diferentes transacciones.
- **Paid with [1:N]:** Una orden puede liquidarse utilizando múltiples transacciones de pago.
- **Receives [1:N]:** Una orden puede recibir múltiples retroalimentaciones o reseñas.
- **Locates [1:N]:** Una zona geográfica concentra a múltiples clientes y múltiples vendedores.

2.3 Restricciones de negocio identificadas.

- **Restricción valor monetario:** Los valores monetarios de precio de producto, costo de flete y valor de pago no pueden tener valores negativos. Deben ser mayores a cero.
- **Restricción reseña:** La calificación otorgada por un cliente en una reseña debe limitarse a un rango numérico entre 1 y 5.
- **Restricción de pago:** El número de cuotas elegidas en un pago debe ser mínimo de 1.
- **Restricción de Dependencia:** Ninguna orden de compra puede existir sin estar vinculada a un cliente.

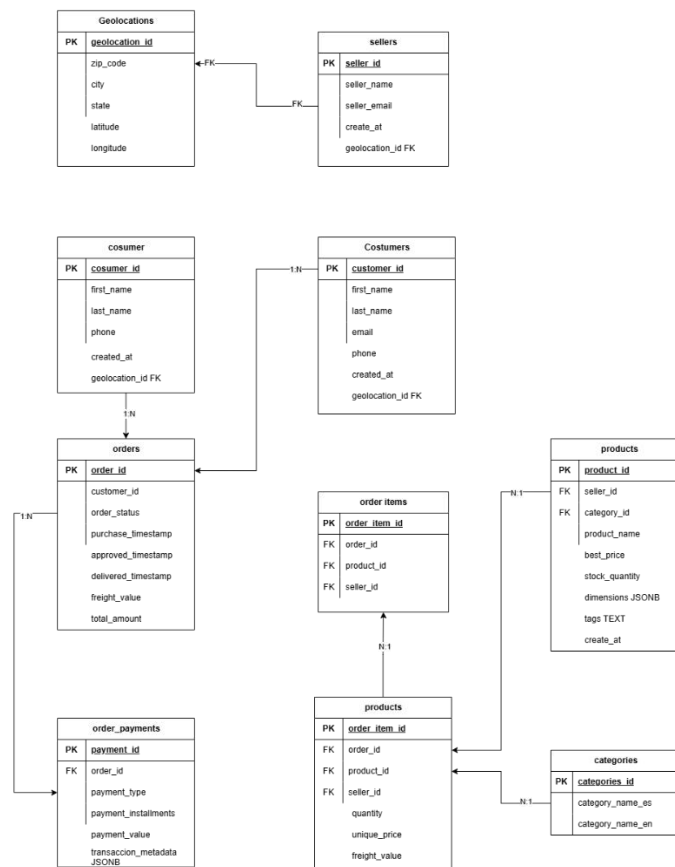
- **Restricción de Dependencia:** Ningún método de pago o reseña puede existir flotando sin estar conectado a una orden válida.

3. Diseño lógico - Módulo PostgreSQL.

Dentro de la definición generada a nivel de arquitectura la base de datos postgres es el motor relacional para usos específico y optimizados como, ordenes, pagos, clientes vendedores y cualquier tipo de información que tenga características relacionales

3.1 Esquema relacional normalizado (3FN)

De acuerdo con el diseño planteado de arquitectura se presenta un modelo normalizado en la tercera forma normal (3FN), esto garantizando integridad, reducción de redundancia, constancia, operatividad y soporte OLTP.



Relación	Cardinalidad
geolocations → customers	1:N
geolocations → sellers	1:N
customers → orders	1:N
orders → order_items	1:N
products → order_items	1:N
sellers → products	1:N
categories → products	1:N
orders → order_payments	1:N

3.2 Tablas

Las siguientes tablas corresponden al modelo transaccional para Postgres de acuerdo con las definiciones anteriormente dadas.

Tabla geolocations: esta tabla almacena la información geográfica para los clientes, el constrain usado es Primary Key (geolocation_id)

Columna	Tipo de dato	Restricción	Descripción
geolocation_id	BIGSERIAL	PRIMARY KEY	Identificador único
zip_code	VARCHAR(20)	NOT NULL	Código postal
city	VARCHAR(100)	NOT NULL	Ciudad
state	VARCHAR(100)	NOT NULL	Estado o región
latitude	NUMERIC(10,6)	NULL	Latitud
longitude	NUMERIC(10,6)	NULL	Longitud

Tabla customers: registra las compras en la Plataforma, constraints usados, primary key(customer ID), UNIQUE (email), Foreign Key (geolocation_id), references geolocations (geolocations_id)

Columna	Tipo de dato	Restricción	Descripción
customer_id	UUID	PRIMARY KEY	Identificador cliente
first_name	VARCHAR (100)	NOT NULL	Nombre
last_name	VARCHAR(100)	NOT NULL	Apellido
email	VARCHAR (150)	UNIQUE NOT NULL	Correo electrónico
phone	VARCHAR (20)	NULL	Teléfono

created_at	TIMESTAMP	DEFAULT NOW ()	Fecha creación
geolocation_id	BIGINT	FOREIGN KEY	Ubicación

Tabla seller: tabla asociada a los vendedores. Constraints usados Primary Key (seller_id), Unique (seller_email), foreign Key (geolocation_id), references geolocations (geolocations_id)

Columna	Tipo de dato	Restricción	Descripción
seller_id	UUID	PRIMARY KEY	Identificador vendedor
seller_name	VARCHAR (150)	NOT NULL	Nombre vendedor
seller_email	VARCHAR(150)	UNIQUE	Correo vendedor
geolocation_id	BIGINT	FOREIGN KEY	Ubicación
created_at	TIMESTAMP	DEFAULT NOW()	Fecha creación

Tabla categorías administra las categorías de productos. Constraints Primary Key (category_id)

Columna	Tipo de dato	Restricción	Descripción
category_id	SERIAL	PRIMARY KEY	Identificador categoría
category_name_es	VARCHAR (120)	NOT NULL	Nombre español
category_name_en	VARCHAR (120)	NOT NULL	Nombre inglés

Tabla products: catalogo de los productos, los datos con información dinámica será administrado por MongoDB. Constraints Primary Key (product_id), UNIQUE (sku), Check (base_price > 0), Check (stock_quantity >= 0), Foreign Key (seller_id), References sellers(seller_id), Foreign Key (category_id), References categories(category_id)

Columna	Tipo de dato	Restricción	Descripción
product_id	UUID	PRIMARY KEY	Identificador producto
seller_id	UUID	FOREIGN KEY NOT NULL	Vendedor
category_id	INTEGER	FOREIGN KEY NOT NULL	Categoría
product_name	VARCHAR(200)	NOT NULL	Nombre producto
base_price	NUMERIC(12,2)	CHECK > 0	Precio base
stock_quantity	INTEGER	CHECK >= 0	Inventario
sku	VARCHAR(100)	UNIQUE	Código SKU
dimensions	JSONB	NULL	Dimensiones dinámicas
tags	TEXT[]	NULL	Etiquetas
created_at	TIMESTAMP	DEFAULT NOW()	Fecha creación

Tabla order: tabla donde se almacenan las compras realizadas. Constraints Primary Key (order_id), Check (freight_value >= 0), Check (total_amount > 0), Foreign Key (customer_id), References customers(customer_id)

Columna	Tipo de dato	Restricción	Descripción
---------	--------------	-------------	-------------

order_id	UUID	PRIMARY KEY	Identificador orden
customer_id	UUID	FOREIGN KEY NOT NULL	Cliente
order_status	VARCHAR(30)	NOT NULL	Estado orden
purchase_timestamp	TIMESTAMP	NOT NULL	Fecha compra
approved_timestamp	TIMESTAMP	NULL	Fecha aprobación
delivered_timestamp	TIMESTAMP	NULL	Fecha entrega
freight_value	NUMERIC(12,2)	CHECK >= 0	Valor envío
total_amount	NUMERIC(12,2)	CHECK > 0	Total orden

Tabla order_items: relaciona los productos de cada orden. Constraints Primary Key (order_id), Check (freight_value >= 0), Check (total_amount > 0), Foreign Key (customer_id), References customers(customer_id)

Columna	Tipo de dato	Restricción	Descripción
order_item_id	BIGSERIAL	PRIMARY KEY	Identificador item
order_id	UUID	FOREIGN KEY NOT NULL	Orden
product_id	UUID	FOREIGN KEY NOT NULL	Producto
seller_id	UUID	FOREIGN KEY NOT NULL	Vendedor
quantity	INTEGER	CHECK > 0	Cantidad
unit_price	NUMERIC(12,2)	CHECK > 0	Precio unitario
freight_value	NUMERIC(10,2)	CHECK >= 0	Valor envío

Tabla order_payments: registra los pagos de la órdenes. Constraints PRIMARY KEY (payment_id), CHECK (payment_installments >= 1), CHECK (payment_value > 0), FOREIGN KEY (order_id). REFERENCES orders(order_id)

Columna	Tipo de dato	Restricción	Descripción
payment_id	BIGSERIAL	PRIMARY KEY	Identificador pago
order_id	UUID	FOREIGN KEY NOT NULL	Orden
payment_type	VARCHAR(50)	NOT NULL	Tipo pago
payment_installments	INTEGER	CHECK >= 1	Número cuotas
payment_value	NUMERIC(12,2)	CHECK > 0	Valor pago
transaction_metadata	JSONB	NULL	Meta data financiera

3.3 Aplicación de tipos avanzados

La definición de arquitectura híbrida entre Postgres y MongoDB para el ecommerce, esa dada por la cantidad de información transaccional, pero también por los atributos dinámicos y datos sin una estructura clara, para esto se pueden utilizar tipos de datos avanzados en postgres como JSONB, arrays

y composite Types. Estos tipos nos ayudaran a tener una flexibilidad entre los esquemas y dando ventajas como integridad rendimiento, consistencia relacional y eficiencia operativa.

JSONB

Como está definido dentro de postgres se pueden almacenar documentos con estructura JSON binaria, dentro de la solución no se busca quitarle esta carga a mongo DB, lo que buscamos realmente es una flexibilidad dentro de campos específicos en el modelo relacional con el fin de tener capacidades controladas desde del modelo relacional.

Ejemplo dentro de nuestra estructura es la tabla products, donde le campo dimensión esta definición JSONB, esto porque es un valor dinámico en cualquier de los elementos que se elijan del ecommerce. El no hacerlo de esta manera genera un nivel de creación de columna pro la dimensión y el tipo del producto generando consulta no optimas y crecimiento exponencial de la tabla en mención.

Otros de los beneficios de JSONB es la generación de consultas rápidas menor cambio en el esquema, integración con MongoDB y un almacenamiento estructura.

Arrays

Desde su función y el tipo de solución a implementar los array son funcionales para almacenar múltiples valores dentro de una columna, facilitando asi su consumo, se relacionamelo, búsqueda y mayor desempeño.

Ejemplo dentro de nuestra estructura es la tabla products, donde el campo que usa ele array es el tag, información del producto para su búsqueda de manera fácil, donde se incluyen características de este.

Composite Types

Su uso está definido para estructura reutilizables que tengan múltiples atributos, en la solución son funcionales para datos que sean usados de manera continua dentro la solicitud de compra o pedido.

Ejemplos dentro de nuestra estructura, serian direcciones de envió, históricos de órdenes, geolocalización, estos datos que se pueden reutilizar durante todo proceso de compra, esto reduce duplicidad, mejor organización y mantenimiento simple.

3.4 Extensiones a utilizar

Con el fin de mejorar aspectos como rendimiento, flexibilidad búsqueda, indexación avanzada entre otras características se realizará el uso de extensiones nativas de postgres para cumplir con estos objetivos.

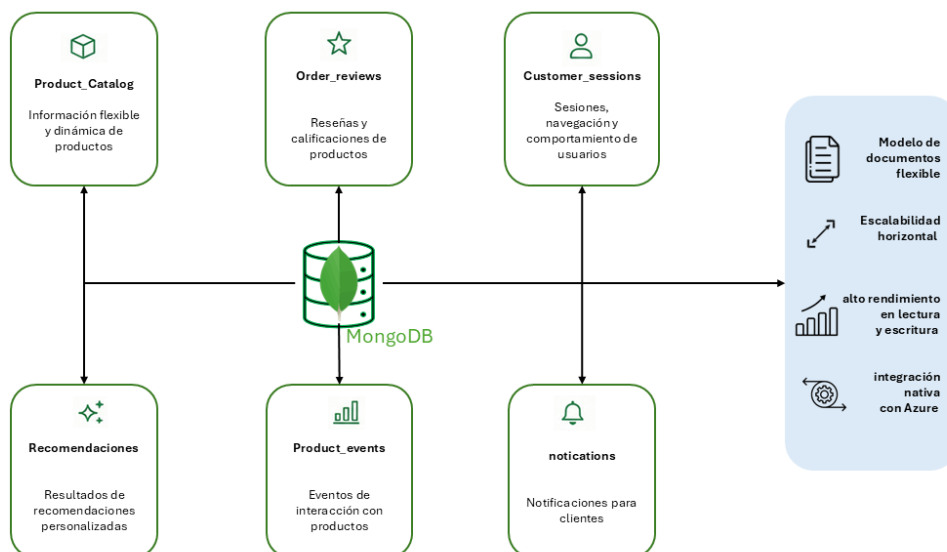
PG_TRGM: Esta extensión nos permite hacer búsquedas de texto sin tener en cuenta errores tipográficos. En la tabla CUSTOMERS en la columna customer_city y en la tabla SELLERS en la columna seller_city, se puede hacer uso de esta extensión para agilizar la consulta de esta información en caso de que por ejemplo haya registro de ciudades que varíen por ejemplo una tilde como sería "sao paulo" contra "são paulo", en este caso la extensión nos permitiría obtener los datos como una misma ciudad.

PG_crypto: esta extensión permite combinar múltiples columnas sin importar el tipo de datos, esto siempre que este dentro de un índice GIN, mejora exponencialmente las consultas complejas donde la búsqueda se realice por varios campos a la vez. Esta es utilizable en la solución planteada de JSONB de productos o en los arrays generados en los tags de productos, dará un mejor resultado en la búsqueda que estén presentes estos cambios y tipo de datos.

• 4. Diseño lógico preliminar - Módulo MongoDB.

4.1 Colecciones principales identificadas

La arquitectura que se construyó define el uso de MongoDB como motor NoSQL, el cual está encargado de administrar información flexible, semiestructurada y de consulta recurrente, de esta manera se completa la gestión dada por postgres.



- **Products_catlog:** administrará el catálogo de productos de manera enriquecida, este será consumido, por aplicaciones web, aplicaciones móviles, motores de búsqueda, relación con la tabla de postgres products.
- **order_reviews:** Gestiona las reseñas y retroalimentación generada por los clientes de acuerdo al producto y compras realizadas, el alto contenido de datos y textos que se pueden dar se aloja en la base de datos de mongo para su correcta gestión, esta almacenara comentarios, calificaciones, imágenes enviadas por los clientes, y otro tipo de metada con iguales funciones.
- **recommendations:** administra las recomendaciones generadas por machine learning y de manera personalizadas para los clientes, de esta manera se puede generar productos sugeridos, afinidad producto-cliente.
- **Product_events:** registra de manera masiva los eventos en los cuales el usuario vendedor interactúa con los productos, estos procesos que se realizan en tiempo real generan un gran volumen en el almacenamiento de registros por eso será utilizado MongoDB.
- **Notification:** administrará todas las notificaciones que se envíen a usuario, como push, correo, estados de pedidos, recordatorios de pedidos, por su volumen se almacenan en MongoDB.
- **Analytics_events:** centraliza datos y métricas de comportamiento de la solución, toma como información principal tráfico, navegación, conversación, tiempos y demás datos que pueda ser actualizados para toma de decisiones.

4.2 Esquema de documentos

A continuación, se presentan ejemplos representativos del esquema de cada colección en formato JSON.

Colección: products

```
json
{
  "_id": "bb50f2a5606a853b3b4337167fa6f49a",
  "product_category_name": "cama_mesa_banho",
  "product_category_name_english": "bed_bath_table",
  "dimensions": {
    "weight_g": 700,
    "length_cm": 30,
    "height_cm": 15,
    "width_cm": 20
  },
  "product_photos_qty": 3,
  "product_name_lenght": 58,
  "product_description_lenght": 1276
}
```

Colección: order_reviews

```
json
{
  "_id": "a54f0611adc9ed256b57ede0a7f60e26",
  "order_id": "e481f51cbdc54678b7cc49136f2d6af7",
  "review_score": 5,
  "review_comment_title": "Produto chegou antes do prazo",
  "review_comment_message": "Excelente vendedor, produto igual ao anunciado.",
  "review_creation_date": "2017-10-12T08:00:00Z",
  "review_answer_timestamp": "2017-10-12T09:30:00Z"
}
```

Colección: geolocation

```

json
{
  "_id": "01310",
  "location": {
    "type": "Point",
    "coordinates": [-46.6560, -23.5616]
  },
  "city": "sao paulo",
  "state": "SP"
}

```

Colección: user_behavior

```

json
{
  "_id": "evt_20171002_001",
  "customer_id": "06b8999e2fba1a1fbc88172c00ba8bc7",
  "event_type": "product_view",
  "product_id": "bb50f2a5606a853b3b4337167fa6f49a",
  "timestamp": "2017-10-02T10:45:00Z",
  "session_id": "sess_abc123",
  "device": "mobile"
}

```

4.3 Patrones de modelado

Debido a que MongoDB trabajara bajo un modelo documental, su diseño de colecciones varia al modelo relacional, por lo cual es necesario aplicar patrones de modelando NoSQL, los cuales permitan generar un mejor rendimiento, escalabilidad, optimización de las consultas, mantenibilidad y esto traducido en una mejor experiencia usuario.

Embedding (documento embebido), Se usa cuando los datos siempre se consultan juntos y no se reutilizan de forma independiente, en products, las dimensiones físicas se embeben dentro del documento del producto porque siempre se necesitan al consultar el catálogo y no existe ningún caso en que se consulten solas.

Bucket (Patrón contenedor), permite organizar y almacenar información agrupándola con datos relacionados o similares dentro de un contenedor o espacio separado. En la a desarrollar este patrón es usado para agrupar eventos continuos y que generan una gran cantidad de data como lo son búsquedas, vistas los productos agregados al carro de compras y todo ese tipo de eventos que de manera natural se ejecutan y que es necesario mantenerlos dentro del sistema.

Reference (referencia por ID) Se usa cuando los datos de una colección son creados y gestionados en otro motor, `order_reviews` se guarda el `order_id` como referencia al registro en PostgreSQL. MongoDB no valida esa referencia, pero la aplicación la usa para cruzar información cuando es necesario. Lo mismo aplica para `customer_id` en `user_behavior`.

Computed (Patrón Calculado), permite tener almacenados resultados precalculados esto optimiza consultas con estos valores repetitivos. Dentro de un ecommerce hay consultas respectivas como venta y popularidad de los productos, recomendaciones, al ser acciones tan repetitivas almacenarlas por medio de este patrón genera las optimizaciones que se buscan, esto genera que una vez se realice nivel de uso de algunos de estos datos se guardadas y actualizados de manera periódica pero no en todo momento lo que genera optimización de las búsquedas.

Schema versioning (Patrón versionado de esquemas): este patrón permite que un mismo documento tenga varias versiones y las alojas dentro de la misma colección, dentro de la arquitectura se aplica, en la categoría a nivel de nuevas o tributos de las actuales, nuevas integraciones, el patrón de versionamiento permite que de esta manera sencilla se genere una adaptación y crecimiento.

Event (patrón de eventos): este patrón está orientado a los eventos generados de la aplicación, nos ayudare dentro de la arquitectura a nivel de eventos de tiempo real con el fin de realizar almacenamiento de datos para tener modelos inteligentes, detectar tendencias, entrenar modelos y más asociados a la experiencia al usuario, adicional a esto permitirá integración otros servicios de manera sencilla como Kafka o big data.

4.4 Justificación de qué datos van a MongoDB vs PostgreSQL

La plataforma Ecommify implementará una arquitectura híbrida utilizando PostgreSQL y MongoDB Atlas, donde cada motor administrará el tipo de información para el cual fue optimizado

PostgreSQL será utilizado para gestionar la información transaccional crítica del eCommerce, incluyendo:

- customers,
- orders,
- order_items,
- order_payments,
- sellers,
- inventory.

Esto se debe a que estos datos requieren:

- integridad relacional,
- consistencia ACID,
- control transaccional,
- trazabilidad financiera.
-

Por otro lado, MongoDB Atlas administrará información flexible, documental y de alto volumen como:

- product_catalog,
- order_reviews,
- customer_sessions,
- product_events,
- recommendations.

Estas colecciones requieren:

- flexibilidad de esquema,
- escalabilidad horizontal,
- almacenamiento semiestructurado,
- consultas rápidas orientadas al frontend.

MongoDB resulta ideal para manejar atributos dinámicos de productos, eventos masivos de navegación, recomendaciones y reseñas con multimedia, evitando modificaciones constantes del esquema relacional. Esta arquitectura híbrida permitirá combinar la consistencia y seguridad de PostgreSQL con la flexibilidad y escalabilidad documental de MongoDB Atlas, optimizando el rendimiento general del eCommerce.

5. Decisiones arquitectónicas justificadas.

5.1 Matriz de decisión por entidad.

Inserta tabla de 6 columnas: **Entidad / Tecnología / Naturaleza del dato / CAP / Operaciones principales / Razón técnica**

- customers / PostgreSQL / Estructurado / C / INSERT, UPDATE, SELECT / Unicidad de identificador, FK desde orders, datos personales que requieren integridad.
- sellers / PostgreSQL / Estructurado / C / INSERT, UPDATE, SELECT / Relaciones con productos y pagos, información fiscal y contractual.
- orders / PostgreSQL / Estructurado / C / INSERT, UPDATE intensivo / Núcleo del negocio. Requiere ACID multi-tabla en un solo commit.
- order_items / PostgreSQL / Estructurado / C / INSERT masivo / Claves foráneas compuestas hacia orders y products, conteos exactos.
- order_payments / PostgreSQL / Estructurado / C / INSERT, UPDATE / Trazabilidad financiera. Necesita consistencia fuerte en operaciones monetarias.
- products / MongoDB / Semi-estructurado / A / find, aggregate masivo / Atributos variables por categoría. Lectura masiva del catálogo.
- order_reviews / MongoDB / Semi-estructurado / A / find masivo, INSERT / Texto libre, calificaciones. Escalabilidad horizontal de lectura.
- geolocation / MongoDB / Semi-estructurado / A / Consultas geoespaciales / Índices 2dsphere, volumen alto con baja transaccionalidad.

- user_behavior / MongoDB / Semi-estructurado / A / INSERT masivo / Esquema cambiante, append-only, sharding por fecha o usuario.

5.2 Análisis del Teorema CAP aplicado.

El Teorema CAP establece que un sistema distribuido solo puede garantizar dos de tres propiedades al mismo tiempo, Consistencia (C), Disponibilidad (A) y Tolerancia a particiones (P).

Para el módulo transaccional en PostgreSQL se prioriza Consistencia sobre Disponibilidad. Operaciones como registrar un pago o actualizar el estado de una orden no pueden arrojar datos desactualizados, si un nodo falla, es preferible que la operación espere o falle limpiamente antes de devolver información incorrecta. PostgreSQL garantiza esto mediante su modelo ACID y niveles de aislamiento configurables.

Para el módulo analítico en MongoDB se prioriza Disponibilidad sobre Consistencia fuerte, consultar el catálogo de productos o las reseñas de un artículo puede tolerar que la información tarde algunos segundos en estar sincronizada entre réplicas, esto es aceptable porque no involucra movimientos de dinero ni estados de órdenes activas, MongoDB maneja esto mediante consistencia eventual en su replica set.

5.3 Trade-offs y estrategias de mitigación.

- Duplicación de identificadores: Claves como product_id y customer_id existen en ambos motores, se acepta como un costo natural de usar dos bases de datos distintas. La aplicación es la responsable de mantener esa coherencia a través del proceso de sincronización.
- Doble modelo de acceso: El backend tiene que conectarse tanto a PostgreSQL como a MongoDB, lo que hace el código un poco más complejo, esto se maneja teniendo bien separado qué parte del sistema le habla a cada motor, evitando que se mezclen las responsabilidades.
- Consistencia eventual en el módulo analítico, una reseña recién publicada puede tardar unos segundos en verse reflejada en todas las réplicas de MongoDB. Esto es aceptable porque no tiene ningún impacto en los pagos ni en el estado de las órdenes activas.

5.4 Flujos de sincronización

Dado que ciertos identificadores de dominio existen en ambos motores, se define la siguiente estrategia para mantener los datos coherentes entre PostgreSQL y MongoDB.

- Dirección: La sincronización es unidireccional, de PostgreSQL hacia MongoDB. PostgreSQL es la fuente única de verdad para los datos críticos.

- Mecanismo: Un job programado consulta los cambios recientes en las tablas relevantes usando un campo `updated_at` y aplica upserts en MongoDB.
- Tolerancia a fallos: Los errores de sincronización se registran en una colección `sync_failures` con reintentos automáticos. Una caída de MongoDB no afecta las operaciones transaccionales en PostgreSQL.
- Punto de control: Se mantiene una tabla `sync_checkpoints` en PostgreSQL con la última fecha de sincronización por entidad, lo que garantiza que el proceso pueda reanudarse sin duplicar registros.

6. Anexos.

6.1 Diccionario de datos completo

A continuación, se describe cada tabla del modelo relacional con sus columnas, tipo de dato, restricciones y descripción funcional.

Inserta tablas de 4 columnas: Columna / Tipo / Restricción / Descripción — una por cada tabla:

Tabla: geolocation

- `geolocation_zip_code_prefix` / VARCHAR(5) / PRIMARY KEY / Código postal. Llave de unión con `customers` y `sellers`.
- `geolocation_lat` / NUMERIC / — / Latitud geográfica.
- `geolocation_lng` / NUMERIC / — / Longitud geográfica.
- `geolocation_city` / VARCHAR(100) / — / Ciudad del código postal.
- `geolocation_state` / CHAR(2) / — / Sigla del estado (ej. SP, RJ).

Tabla: customers

- `customer_id` / VARCHAR(32) / PRIMARY KEY / Identificador único del cliente por transacción.
- `customer_unique_id` / VARCHAR(32) / NOT NULL / Identificador persistente del cliente entre transacciones.
- `customer_zip_code_prefix` / VARCHAR(5) / FK → `geolocation` / Código postal de residencia.
- `customer_city` / VARCHAR(100) / — / Ciudad del cliente.
- `customer_state` / CHAR(2) / — / Estado del cliente.
- `contact_cellphones` / TEXT[] / — / Arreglo de teléfonos. Tipo ARRAY.

Tabla: sellers

- `seller_id` / VARCHAR(32) / PRIMARY KEY / Identificador único del vendedor.
- `seller_zip_code_prefix` / VARCHAR(5) / FK → `geolocation` / Código postal de operación.
- `seller_city` / VARCHAR(100) / — / Ciudad del vendedor.
- `seller_state` / CHAR(2) / — / Estado del vendedor.
- `contact_cellphones` / TEXT[] / — / Arreglo de teléfonos. Tipo ARRAY.

Tabla: orders

- `order_id` / VARCHAR(32) / PRIMARY KEY / Identificador único de la orden.
- `customer_id` / VARCHAR(32) / FK → `customers`, NOT NULL / Cliente que realizó la compra.

- order_status / VARCHAR(50) / NOT NULL / Estado: created, approved, shipped, delivered, canceled.
- order_purchase_timestamp / TIMESTAMP / — / Fecha de creación de la orden.
- order_approved_at / TIMESTAMP / — / Fecha de aprobación del pago.
- order_delivered_carrier_date / TIMESTAMP / — / Fecha de entrega al transportista.
- order_delivered_customer_date / TIMESTAMP / — / Fecha de entrega al cliente.
- order_estimated_delivery_date / TIMESTAMP / — / Fecha estimada de entrega.
- order_metadata / JSONB / — / Respuesta de pasarela de pagos. Tipo JSONB.

Tabla: order_items

- order_id / VARCHAR(32) / PK compuesta, FK → orders / Referencia a la orden padre.
- order_item_id / INTEGER / PK compuesta / Número secuencial del ítem en la orden.
- product_id / VARCHAR(32) / FK → products / Producto del ítem.
- seller_id / VARCHAR(32) / FK → sellers / Vendedor responsable.
- shipping_limit_date / TIMESTAMP / — / Fecha límite de despacho.
- price / money_amount / CHECK amount >= 0 / Precio del producto. Composite type.
- freight_value / money_amount / CHECK amount >= 0 / Costo de flete. Composite type.

Tabla: order_payments

- order_id / VARCHAR(32) / PK compuesta, FK → orders / Referencia a la orden pagada.
- payment_sequential / INTEGER / PK compuesta / Número secuencial del pago.
- payment_type / VARCHAR(50) / NOT NULL / Método: credit_card, boleto, voucher, debit_card.
- payment_installments / INTEGER / CHECK >= 1 / Número de cuotas. Mínimo 1.
- payment_value / money_amount / CHECK amount >= 0 / Valor del pago. Composite type.

Tabla: order_reviews

- review_id / VARCHAR(32) / PK compuesta / Identificador único de la reseña.
- order_id / VARCHAR(32) / PK compuesta, FK → orders / Orden que recibe la reseña.
- review_score / INTEGER / CHECK BETWEEN 1 AND 5 / Calificación del 1 al 5.
- review_comment_title / VARCHAR(255) / — / Título opcional del comentario.
- review_comment_message / TEXT / — / Cuerpo del comentario.
- review_creation_date / TIMESTAMP / — / Fecha de creación de la reseña.
- review_answer_timestamp / TIMESTAMP / — / Fecha de respuesta de la plataforma.

6.2 Scripts SQL preliminares

Los siguientes scripts corresponden al modelo relacional propuesto para PostgreSQL incluyendo los tipos avanzados del diseño lógico.

```

sql

-- Composite type para manejo multi-moneda
CREATE TYPE money_amount AS (
    amount    NUMERIC(12,2),
    currency  CHAR(3)
);

-- Tabla: geolocation
CREATE TABLE geolocation (
    geolocation_zip_code_prefix VARCHAR(5)    PRIMARY KEY,
    geolocation_lat              NUMERIC,
    geolocation_lng              NUMERIC,
    geolocation_city              VARCHAR(100),
    geolocation_state             CHAR(2)
);

-- Tabla: customers
CREATE TABLE customers (
    customer_id                  VARCHAR(32)    PRIMARY KEY,
    customer_unique_id           VARCHAR(32)    NOT NULL,
    customer_zip_code_prefix     VARCHAR(5)     NOT NULL,
    customer_city                 VARCHAR(100),
    customer_state                CHAR(2),
    contact_cellphones            TEXT[],
    CONSTRAINT fk_customer_geo FOREIGN KEY (customer_zip_code_prefix)
        REFERENCES geolocation(geolocation_zip_code_prefix)
);

```

```
-- Tabla: sellers
CREATE TABLE sellers (
    seller_id          VARCHAR(32) PRIMARY KEY,
    seller_zip_code_prefix VARCHAR(5) NOT NULL,
    seller_city        VARCHAR(100),
    seller_state       CHAR(2),
    contact_cellphones TEXT[],
    CONSTRAINT fk_seller_geo FOREIGN KEY (seller_zip_code_prefix)
        REFERENCES geolocation(geolocation_zip_code_prefix)
);

-- Tabla: orders
CREATE TABLE orders (
    order_id          VARCHAR(32) PRIMARY KEY,
    customer_id       VARCHAR(32) NOT NULL,
    order_status      VARCHAR(50) NOT NULL,
    order_purchase_timestamp TIMESTAMP,
    order_approved_at  TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP,
    order_metadata     JSONB,
    CONSTRAINT fk_order_customer FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

```
-- Tabla: order_items
CREATE TABLE order_items (
    order_id          VARCHAR(32) NOT NULL,
    order_item_id     INTEGER      NOT NULL,
    product_id        VARCHAR(32) NOT NULL,
    seller_id         VARCHAR(32) NOT NULL,
    shipping_limit_date TIMESTAMP,
    price             money_amount NOT NULL,
    freight_value     money_amount NOT NULL,
    PRIMARY KEY (order_id, order_item_id),
    CONSTRAINT fk_item_order FOREIGN KEY (order_id)
        REFERENCES orders(order_id),
    CONSTRAINT fk_item_seller FOREIGN KEY (seller_id)
        REFERENCES sellers(seller_id)
);

-- Tabla: order_payments
CREATE TABLE order_payments (
    order_id          VARCHAR(32) NOT NULL,
    payment_sequential INTEGER      NOT NULL,
    payment_type       VARCHAR(50) NOT NULL,
    payment_installments INTEGER     NOT NULL CHECK (payment_installments >= 1),
    payment_value      money_amount NOT NULL,
    PRIMARY KEY (order_id, payment_sequential),
    CONSTRAINT fk_payment_order FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
);
```

```
-- Tabla: order_reviews
CREATE TABLE order_reviews (
    review_id          VARCHAR(32) NOT NULL,
    order_id           VARCHAR(32) NOT NULL,
    review_score        INTEGER      NOT NULL CHECK (review_score BETWEEN 1 AND 5),
    review_comment_title VARCHAR(255),
    review_comment_message TEXT,
    review_creation_date TIMESTAMP,
    review_answer_timestamp TIMESTAMP,
    PRIMARY KEY (review_id, order_id),
    CONSTRAINT fk_review_order FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
);

-- Extensión búsqueda tolerante a errores tipográficos
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_customers_city_trgm ON customers
    USING gin (customer_city gin_trgm_ops);

CREATE INDEX idx_sellers_city_trgm ON sellers
    USING gin (seller_city gin_trgm_ops);
```

6.3 Ejemplos de datos

```
-- geolocation INSERT INTO geolocation VALUES ('01310', -23.5616, -46.6560, 'sao paulo', 'SP'), ('20040',
-22.9068, -43.1729, 'rio de janeiro', 'RJ');
-- customers con ARRAY de teléfonos INSERT INTO customers VALUES
('06b8999e2fba1a1fbc88172c00ba8bc7', '861eff4711a542e4b93843c6dd7febb0', '01310', 'sao paulo',
'SP', ARRAY['+55 11 91234-5678', '+55 11 98765-4321']);
-- sellers con ARRAY de teléfonos INSERT INTO sellers VALUES ('3442f8959a84dea7ee197c632cb2df15',
'01310', 'sao paulo', 'SP', ARRAY['+55 11 93333-4444']);
-- orders con JSONB de pasarela de pagos INSERT INTO orders VALUES
('e481f51cbdc54678b7cc49136f2d6af7', '06b8999e2fba1a1fbc88172c00ba8bc7', 'delivered', '2017-10-
02 10:56:33', '2017-10-02 11:07:15', '2017-10-04 19:55:00', '2017-10-10 21:25:13', '2017-10-18
00:00:00', '{"gateway":"pagarme","status":"paid", "transaction_id":"1234567", "message":"Transação
aprovada"}::jsonb');
-- order_items con composite type INSERT INTO order_items VALUES
('e481f51cbdc54678b7cc49136f2d6af7', 1, 'bb50f2a5606a853b3b4337167fa6f49a',
'3442f8959a84dea7ee197c632cb2df15', '2017-10-06 11:07:15', ROW(29.99, 'BRL')::money_amount,
ROW(8.72, 'BRL')::money_amount);
-- order_payments con composite type INSERT INTO order_payments VALUES
('e481f51cbdc54678b7cc49136f2d6af7', 1, 'credit_card', 1, ROW(38.71, 'BRL')::money_amount);
-- order_reviews INSERT INTO order_reviews VALUES ('a54f0611adc9ed256b57ede0a7f60e26',
'e481f51cbdc54678b7cc49136f2d6af7', 5, 'Produto chegou antes do prazo', 'Excelente vendedor,
produto igual ao anunciado.', '2017-10-12 08:00:00', '2017-10-12 09:30:00');
```